

Lab — Scaling Laws on DarijaDZ

Basics of scaling laws, applied to an Algerian Darija corpus

Contents

Lab: Scaling Laws on DarijaDZ	1
0. Motivation	1
1. Learning objectives	1
2. Background you'll be leaning on (from the lecture)	2
3. Environment and data	2
4. Part 1 — Data scaling law	3
5. Part 2 — Model scaling law	3
6. Part 3 — Joint scaling and the compute-optimal frontier	3
7. Part 4 — Tokenizer vocabulary as a scaling axis	4
8. Part 5 — Discussion (write-up, no new runs required)	4
9. Deliverables	5
10. Stretch goals (optional, not required for the core lab)	5

Lab: Scaling Laws on DarijaDZ

*Companion lab to CS336 Lecture 9 (“Scaling Laws — Basics”). Where the lecture fits transformers, LSTMs, and MT/speech data from Kaplan, Hestness, Hoffmann (Chinchilla) et al., this lab asks the same questions of **DarijaDZ**, the project’s own 16.2M-document Algerian Darija corpus — a low-resource, code-switched, multi-script setting quite unlike the English web-text these classic papers were fit on.*

0. Motivation

Every scaling-law paper in the lecture was fit on a resource-rich language (English) with a huge, roughly homogeneous corpus. DarijaDZ is the opposite case: ~214.7M word-level tokens total (small by LLM-pretraining standards), short documents (mean **13.25 tokens/doc** — these are comments, not articles), and a document population that is a genuine mixture of scripts and languages (70.3% Darija/Arabic-script, 9.6% Arabizi/Latin-script, 9.2% MSA, 6.1% French, 3.1% code-switched, 1.2% English, 0.5% junk — see `DarijaDZ/README.md`).

The lecture’s central claim is that scaling laws are *engineered* regularities, not automatic ones — the exponents you get depend on how you count parameters, how you tokenize, how converged your small runs are, and what corpus you’re even fitting on. This lab tests that claim directly: do the classic exponents (data ~ −0.05 to −0.1, joint N/D split ~ 0.5/0.5) show up on a corpus that is two orders of magnitude smaller and structurally different from what Kaplan/Hoffmann used? If not, why not?

1. Learning objectives

By the end of this lab you should be able to:

1. Fit a **data scaling law** (loss vs. dataset size) on **log-log axes** and extract the **exponent**, the way Section “Data vs performance” of the lecture does for Kaplan/Hestness/Banko-Brill.
2. Fit a **model scaling law** (loss vs. non-embedding parameters) and discuss why “non-embedding” is the operative word (lecture slide “Depth/Width: But not all parameters are made equal”).
3. Run an **IsoFLOPs sweep** (lecture’s “Method 2”) to find a compute-optimal frontier $N^*(C)$, $D^*(C)$ for DarijaDZ, fit its exponents, and compare them to Kaplan’s (0.73, 0.27) and Chinchilla’s (~0.5, 0.5).
4. Treat **tokenizer vocabulary size** as its own scaling axis using the project’s existing BPE/WordPiece/SentencePiece(Unicode) tokenizers at 5 vocab budgets each, and connect it to the embedding-parameter discussion.
5. Reason about where a **finite, structurally mixed** corpus should be expected to break the standard scaling story (repetition limits, data composition/offset-vs-slope effects, upstream-vs-downstream transfer).

2. Background you’ll be leaning on (from the lecture)

- **Power-law form:** $\log(\text{loss}) = -\alpha * \log(x) + C$ is linear on a log-log plot; x can be data size, non-embedding parameters, or compute.
- **Data scaling law origin:** for the mean-estimation toy example, $E[(\mu_{\text{hat}} - \mu)^2] = \sigma^2 / n$, i.e. exponent -1 ; for d -dimensional nonparametric regression, exponent $-1/d$. Real neural LM data-scaling exponents (Kaplan: ~ -0.095) are much shallower than the parametric -1 case — a sign that LMs behave more like high-dimensional nonparametric estimators than simple parametric ones.
- **Not all parameters are equal:** Kaplan et al. exclude both embedding *and* last-layer parameters when fitting N ; the Wortsman et al. (“Resolving Discrepancies...”) and Pearce & Song papers trace much of the Kaplan/Chinchilla disagreement to exactly this choice, plus LR warmup and batch-size tuning at small scale.
- **Joint scaling law** (Rosenfeld): $\text{Error} = n^{-\alpha} + m^{-\beta} + C$; or (Kaplan): $\text{Error} = [m^{-\alpha} + n^{-1}]^{\beta}$. Both fit well when extrapolated from small (n, m) to large.
- **Compute-optimal (“Chinchilla”) procedure:** 3 independent methods —
 (1) lower envelope of training curves, (2) IsoFLOPs profiles, (3) direct parametric fit of $L(N, D)$ — that should broadly agree if your setup is well-tuned.
- **Slopes vs. intercepts:** across almost every intervention discussed in the lecture (data mixture, SGD vs Adam, repetition, regularization), the *slope* of the scaling law is remarkably stable — interventions mostly shift the *intercept*. A key thing to check in your own results.
- **Repetition is (almost) free up to ~4 epochs**, then rapidly worthless (Muennighoff et al., “Scaling Data-Constrained Language Models”) — directly relevant here since DarijaDZ is a *finite* corpus.
- **Upstream != downstream:** perplexity scaling is clean; task-accuracy scaling is not guaranteed to follow the same ranking (Tay et al. 2023, NL12 vs NL32-XL). Keep this in mind before you claim a “better” recipe.

3. Environment and data

- Use the **GPU** venv for anything importing torch (per the project’s CLAUDE.md): `...\deep learning\labs\training neural networks\ai-gpu\Scripts\python.exe`. The base env’s torch build is CPU-only.
- **Corpus:** `DarijaDZ/youtube_corpus.jsonl` — 16,197,229 lines, each `{"id": ..., "text": ...}`, already cleaned (URL/mention placeholders, diacritics stripped, near-dup removed via MinHash/LSH, near-empty dropped). ~3 GB on disk. Docs are short and comment-shaped — you will need to **pack multiple documents per training sequence** (with a separator/EOS token) rather than truncate/pad one doc per sequence, or your **effective context length will be mostly padding**.
- **Tokenizers already trained** in `Tokenization/models/` — do not retrain these unless a task asks you to:
 - `bpe/bpe_{1000,5000,10000,20000,30000}`
 - `wordpiece/wordpiece_{1000,5000,10000,20000,30000}`
 - `sentencepiece/unigram_{1000,5000,10000,20000,30000}.model` (and a larger `darija_unigram.model` — check its vocab size before using it as a “6th” point on any vocab-size plot, it wasn’t trained at the same budget as the others).

- **Held-out set:** `Tokenization/data/heldout_docs.jsonl` already exists for tokenizer eval; reuse it (or **carve your own held-out** split from the corpus, seeded, disjoint from anything used to fit a model) as your loss eval set for Parts 1–3 so results across parts are comparable.
- Keep runs small on purpose — this is a **“fit a few small models, extrapolate”** lab in the spirit of the lecture’s own framing (§ “Old and unpleasant vs. new (over?) optimism” slide), not a production training run. Budget your compute up front: decide how many total **GPU-hours you have**, and divide it across Parts 1–4 *before* you start, the same way you’d have to on a real scaling-law study.

4. Part 1 — Data scaling law

Question: How does DarijaDZ’s next-token loss scale with training data size, and how does the exponent compare to the lecture’s reference values?

1. Fix a small causal **LM architecture** (decoder-only transformer, e.g. `d_model=128, n_layer=4, n_head=4`, using the `bpe_10000` tokenizer) — small enough that even your largest data point trains in minutes, per the “model » data” rule of thumb from the transcript’s Q&A (**aim for the model to stay well above the data-limited regime until your biggest D**).
2. Build 5–6 training subsets at geometrically spaced sizes, e.g. roughly **{100K, 300K, 1M, 3M, 10M, 30M}** tokens, sampled from disjoint prefixes of a **shuffled corpus** (reuse the shuffling logic from `Youtube_scrap/scripts/build_unified_dataset.py` if convenient, or a fresh seeded shuffle of `youtube_corpus.jsonl` — either way, document which one you used).
3. **Train the same model config from scratch on each subset** for a fixed token budget or until the loss curve visibly flattens (your call — state which and why). Record **final held-out loss** for each.
4. Plot **$\log(\text{tokens})$ vs $\log(\text{held-out loss})$** . **Fit a line**; report the **slope (exponent)** and its confidence interval.
5. **Compare:** how does your exponent compare to Kaplan’s ~ -0.095 for English LM data scaling? If it’s noticeably different (likely, given corpus size and script mixture), propose at least two candidate explanations grounded in what’s specific to DarijaDZ (short/noisy documents, multi-script mixture, far smaller total corpus than Kaplan’s training range, etc.) rather than just “small-scale noise.”

5. Part 2 — Model scaling law

Question: How does loss scale with non-embedding parameter count, and does the embedding-vs-non-embedding distinction (lecture slide “But not all parameters are made equal”) actually matter at DarijaDZ’s scale?

1. Fix a **data budget large enough that all your model sizes stay in the power-law** (not data-limited) regime — use your Part 1 results to justify the choice (e.g. “the loss curve at 10M tokens was still power-law-shaped for my Part-1 model, so I’ll fix $D = 10\text{M tokens here}$ ”).
2. Train **4–5 models varying width/depth** at a **roughly fixed aspect ratio** (d_model / n_layer , per the lecture’s aspect-ratio slide) so you’re not confounding “bigger” with “differently shaped.” Use `bpe_10000` again to hold vocabulary fixed.
3. For each model, record loss two ways: against **total parameters**, and against **non-embedding** parameters (i.e. subtract `vocab_size * d_model` — and, if your output layer isn’t tied to the input embedding, subtract that too). Plot both on the same log-log axes, same style as the lecture’s “Parameters (with embedding)” vs “Parameters (non-embedding)” comparison.
4. Report both fitted exponents. At what parameter scale does the choice of axis start to visibly change the fit? Given DarijaDZ’s small vocab budgets (1K–30K, versus GPT-2’s 50K), is the embedding-parameter fraction here larger or smaller than in a typical English LM of the same `d_model`, and does that make the non-embedding-vs-total distinction more or less of a big deal for this corpus?

6. Part 3 — Joint scaling and the compute-optimal frontier

Question: For a fixed compute budget, how should DarijaDZ pretraining compute be split between model size and data — and does the answer look more like Kaplan’s N is proportional to $C^{0.73}$ or Chinchilla’s

N is proportional to $C^{0.5}$?

1. Pick 3–4 FLOP budgets spanning roughly 2 orders of magnitude (small, on purpose — you are not trying to reproduce Chinchilla’s compute range, you’re reproducing its *method* at a scale your GPU venv can actually run). Approximate FLOPs with the standard $C \sim 6 N D$ rule used throughout the lecture.
2. For each budget, sweep model size (implying token count via the fixed FLOP constraint) and train each point to completion — this is the lecture’s **Method 2 (IsoFLOPs)**. Plot loss vs. parameters per budget; each curve should be convex with a visible minimum, like the lecture’s IsoFLOPs figure.
3. Take the minimum of each curve, plot $N^*(C)$ and $D^*(C)$ on log-log axes, fit power laws, and report the exponents a , b where N^* is proportional to C^a , D^* is proportional to C^b .
4. **Sanity check against repetition limits:** at your largest FLOP budget, what is the implied D^* in tokens, and how many epochs over DarijaDZ’s 214.7M tokens does that represent? If it’s pushing past ~ 4 epochs, flag it explicitly — per the “Scaling Data-Constrained Language Models” result from the lecture, repetition beyond that point gives rapidly diminishing (and eventually $\sim$ worthless) returns, which would bias your fitted $D^*(C)$ exponent if you don’t account for it.
5. **Discuss:** DarijaDZ is $\sim 1000\times$ smaller than the corpora Kaplan/Chinchilla fit on. Given the repetition-limit finding above, would you expect a *token-scarce* corpus like this to push the compute-optimal split toward more parameters and fewer (repeated) tokens, or does the repetition penalty argue the other way? Use your own numbers, not just the lecture slides, to make the case.

7. Part 4 — Tokenizer vocabulary as a scaling axis

Question: The project already trained 3 tokenizer families (BPE, WordPiece, SentencePiece-Unigram) at 5 vocab budgets each (1K/5K/10K/20K/30K). Does tokenizer choice interact with scaling the way the lecture’s “many architectures” (Tay et al.) or “SGD vs Adam” (Hestness) comparisons showed — different intercept, similar slope? Or does it move the slope too?

1. For each of the 15 (family $\times$ vocab-size) tokenizers, compute **fertility** (tokens per word) and **compression** (chars per token) on the held-out set — reuse the metric code from `Tokenization/tokenization_eval.ipynb` rather than rederiving it.
2. Plot fertility vs. vocab size (log-x) for each of the 3 families on one chart — 3 curves, 5 points each. Does fertility keep dropping smoothly, or does it plateau by 10K–20K for this corpus? (Recall DarijaDZ is multi-script and only ~ 214.7 M tokens total — a 30K-vocab tokenizer trained on that little data may already be over-parameterized for its training set in a way a 50K English BPE tokenizer trained on billions of tokens would not be.)
3. Pick your best-performing vocab size from step 2 and re-run a *small* slice of Part 1 (2–3 data points is enough) with each of the 3 tokenizer families at that fixed vocab size, same model config otherwise. Does tokenizer family shift the loss-vs-data curve’s **intercept**, its **slope**, or both? (Careful: cross-tokenizer loss values are *not* directly comparable per-token — normalize by bits-per-character or bits-per-word before comparing, and say explicitly which normalization you used.)
4. Connect back to Part 2: for your smallest model (`d_model=128`) at `vocab_size=30000`, what fraction of total parameters is the embedding table? At what `d_model` would that fraction drop below, say, 10%? This is the DarijaDZ-specific version of the lecture’s “embedding parameters don’t behave the same” caution.

8. Part 5 — Discussion (write-up, no new runs required)

Answer these in your write-up, citing your own Part 1–4 numbers where relevant, not just lecture slides:

1. **Slopes vs. intercepts.** Across Parts 1, 3, and 4, which of your interventions shifted the fitted exponent (slope) vs. just the intercept? The lecture repeatedly found slopes to be strikingly stable under intervention (SGD $\rightarrow$ Adam, data mixture, regularization) — did you find the same, or does DarijaDZ’s small-corpus regime behave differently?
2. **Upstream vs. downstream.** You’ve only measured held-out next-token loss. `Dialect_Identification/`, `LM_DiD/`, and the `Benchmarks` folders in this repo have downstream tasks available. If you had time

to check one, which would you pick to test whether your Part-1/Part-2 loss-scaling ranking of models actually predicts downstream ranking — and why that one, given the lecture’s NL12-vs-NL32-XL warning that these can diverge?

3. **Data composition, not just size.** DarijaDZ’s `README.md` breaks the corpus into Darija/MSA/Arabizi/French/English switched/junk percentages, but the `script/darija_confidence` fields described in the project’s `CLAUDE.md` as “not yet built” (Phase 1 language/dialect filter) don’t exist yet. If they did, how would you set up a distribution-shift-style ablation (lecture’s Hashimoto 2021 slide — “data composition affects the offset, not the slope”) using those fields, e.g. Darija-only vs. Darija+Arabizi vs. everything? What result would convince you the extra scripts are worth keeping vs. filtering out?
4. **The Kaplan/Chinchilla discrepancy, applied to yourself.** The lecture traced most of the Kaplan-vs-Chinchilla gap to 3 causes: embedding-param exclusion, LR warmup at tiny scale, and untuned batch size. Audit your own Part 3 setup against these three. Which did you control for, which didn’t you, and how would you expect each uncontrolled one to bias your fitted **a**, **b** exponents specifically?

9. Deliverables

- One notebook per part (or one combined notebook with clearly labeled sections) under `DarijaDZ_scalinglaws/notebooks/`, saved with outputs.
- 4 log-log plots (Parts 1, 2, 3, 4) with fitted exponents and their confidence intervals annotated on the plot itself, not just in prose.
- A results table: for every fitted exponent in the lab, list your value, its CI, and the nearest lecture/paper reference value for comparison (Kaplan data-scaling ~ -0.095 , Kaplan **(a,b)** = (0.73, 0.27), Chinchilla **(a,b)** $\sim$ (0.5, 0.5), etc.).
- A $\sim$ 1-page write-up answering Part 5’s four discussion questions.

10. Stretch goals (optional, not required for the core lab)

- **Critical batch size:** pick one model/data config from Part 1 and sweep batch size at a fixed target loss, fit `B_crit` per the lecture’s $S/S_{\min} - 1 = (E/E_{\min} - 1)^{-1}$ relation, and check whether `B_crit` grows as loss decreases the way the lecture’s slide predicts.
- **Repetition scaling:** deliberately over-repeat a small (e.g. 1M-token) subset for 1/4/16/64 epochs and reproduce the “repeating is almost free up to ~ 4 epochs, then worthless” shape from Muennighoff et al. on DarijaDZ’s own text instead of C4/English data.
- **Script-conditional scaling:** split held-out loss by script (Arabic vs. Latin vs. mixed, using a cheap regex-based script classifier — the same logic already used by `build_unified_dataset.py`’s script-distribution table) and check whether the Part 1 data-scaling exponent differs by script. A meaningfully different exponent per script would be a concrete argument for (or against) per-script data curation.